

Task2:Data cleaning and Exploratory Data Analysis(EDA)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data=pd.read_csv('/content/online shopping.csv')
data.head()
```

	Unnamed: 0	CustomerID	Gender	Location	Tenure_Months	Transaction_ID	Transaction_Date	Product_SKU	Product_Description
0	0	17850.0	M	Chicago	12.0	16679.0	1/1/2019	GGOENEBJ079499	Nest Learning Thermostat 3rd Gen-USA - Stainle...
1	1	17850.0	M	Chicago	12.0	16680.0	1/1/2019	GGOENEBJ079499	Nest Learning Thermostat 3rd Gen-USA - Stainle...
2	2	17850.0	M	Chicago	12.0	16696.0	1/1/2019	GGOENEBQ078999	Nest Cam Outdoor Security Camera - USA

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52924 entries, 0 to 52923
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            52924 non-null  int64
1   CustomerID                            52924 non-null  int64
2   Gender                                52924 non-null  object
3   Location                               52924 non-null  object
4   Tenure_Months                         52924 non-null  int64
5   Transaction_ID                         52924 non-null  int64
6   Transaction_Date                       52924 non-null  object
7   Product_SKU                           52924 non-null  object
8   Product_Description                   52924 non-null  object
9   Product_Category                     52924 non-null  object
10  Quantity                              52924 non-null  int64
```

```
data.describe()
```

	Unnamed: 0	CustomerID	Tenure_Months	Transaction_ID	Quantity	Avg_Price	Delivery_Charges	GST	Offline_Spend	Online_Sper
count	52924.00000	52924.00000	52924.00000	52924.00000	52924.00000	52924.00000	52924.00000	52924.00000	52924.00000	52924.00000
mean	26461.50000	15346.70981	26.127995	32409.825675	4.497638	52.237646	10.517630	0.137462	2830.914141	1893.10911
std	15277.98716	1766.55602	13.478285	8648.668977	20.104711	64.006882	19.475613	0.045825	936.154247	807.01409
min	0.00000	12346.00000	2.000000	16679.00000	1.000000	0.390000	0.000000	0.050000	500.000000	320.25000
25%	13230.75000	13869.00000	15.000000	25384.00000	1.000000	5.700000	6.000000	0.100000	2500.000000	1252.63000
50%	26461.50000	15311.00000	27.000000	32625.50000	1.000000	16.990000	6.000000	0.180000	3000.000000	1837.87000
75%	39692.25000	16996.25000	37.000000	39126.25000	2.000000	102.130000	6.500000	0.180000	3500.000000	2425.35000

```
data.columns
```

```
Index(['Unnamed: 0', 'CustomerID', 'Gender', 'Location', 'Tenure_Months',
      'Transaction_ID', 'Transaction_Date', 'Product_SKU',
      'Product_Description', 'Product_Category', 'Quantity', 'Avg_Price',
      'Delivery_Charges', 'Coupon_Status', 'GST', 'Date', 'Offline_Spend',
      'Online_Spend', 'Month', 'Coupon_Code', 'Discount_pct'],
      dtype='object')
```

```
data.isnull().sum()
```

	0
Unnamed: 0	0
CustomerID	0
Gender	0
Location	0
Tenure_Months	0
Transaction_ID	0
Transaction_Date	0
Product_SKU	0
Product_Description	0

Handle Missing Values

```
data['Coupon_Code'].fillna('No Coupon', inplace=True)  
data['Discount_pct'].fillna(0, inplace=True)
```

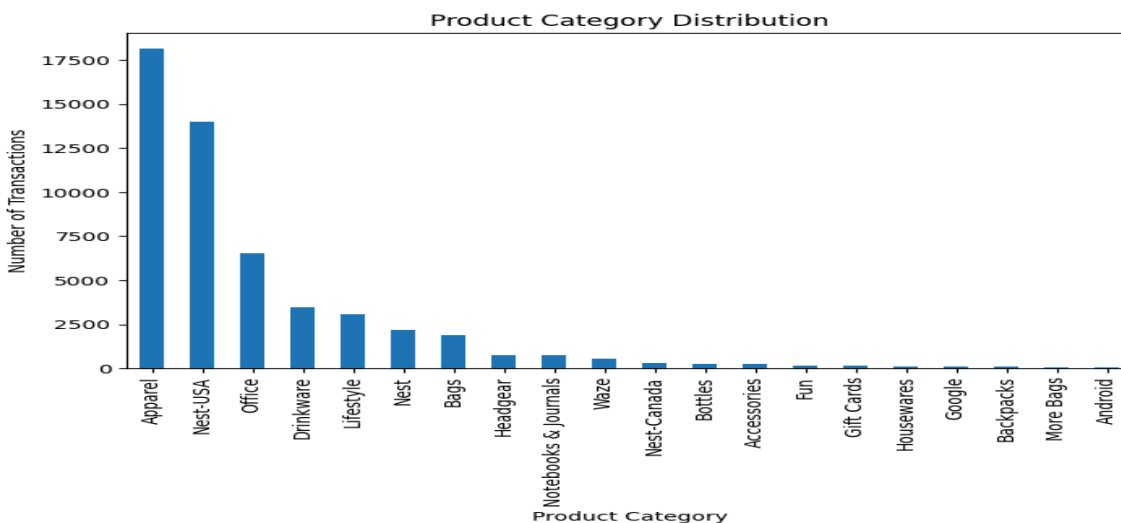
/tmp/ipython-input-3911400960.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series consisting of zero or more existing rows. The behavior will change in pandas 3.0. This inplace method will never work.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df[col] = df[col].method(value)' instead.

Remove Duplicate Records

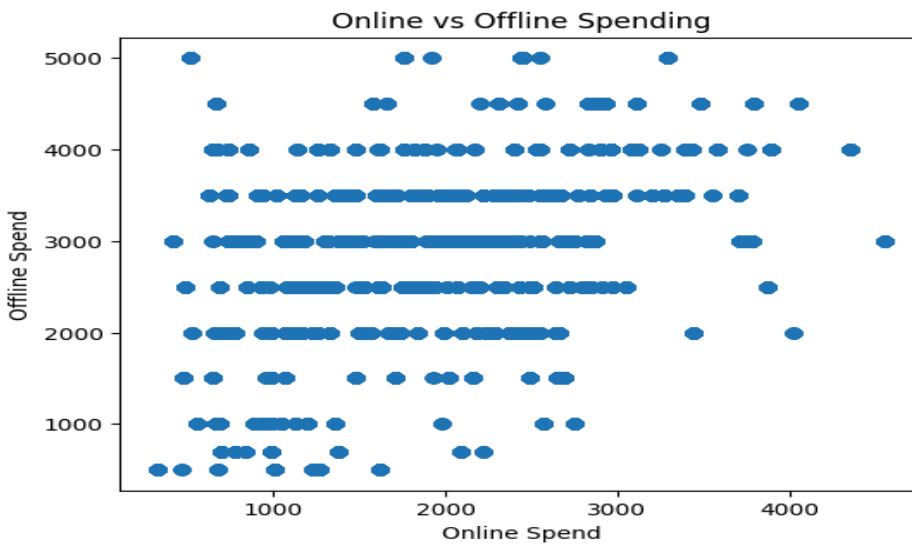
```
data.drop_duplicates(inplace=True)
```

Exploratory Data Analysis (EDA) :Gender Distribution (Categorical Analysis)

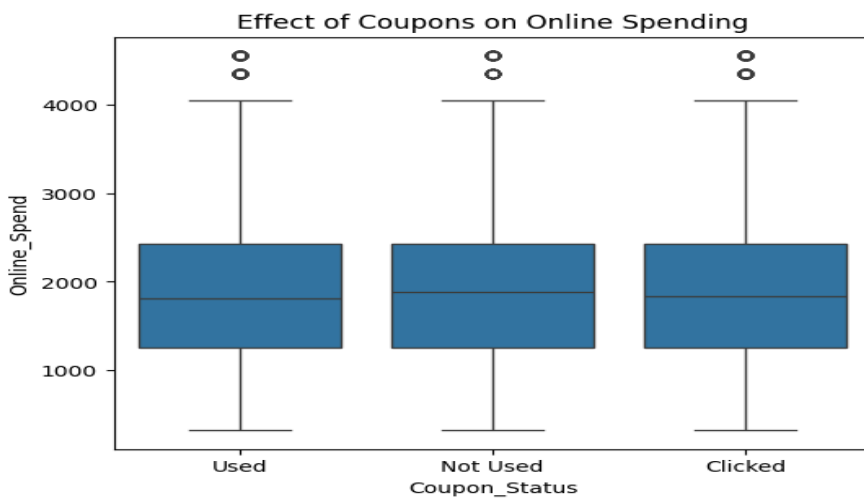
```
plt.figure(figsize=(8,5))  
data['Product_Category'].value_counts().plot(kind='bar')  
plt.xlabel("Product Category")  
plt.ylabel("Number of Transactions")  
plt.title("Product Category Distribution")  
plt.show()
```



```
plt.figure(figsize=(6,5))
plt.scatter(data['Online_Spend'], data['Offline_Spend'])
plt.xlabel("Online Spend")
plt.ylabel("Offline Spend")
plt.title("Online vs Offline Spending")
plt.show()
```



```
plt.figure(figsize=(6,5))
sns.boxplot(x='Coupon_Status', y='Online_Spend', data=data)
plt.title("Effect of Coupons on Online Spending")
plt.show()
```



```
plt.figure(figsize=(8,6))
sns.heatmap(data[['Quantity', 'Avg_Price', 'Discount_pct',
                  'Online_Spend', 'Offline_Spend']].corr(),
            annot=True, cmap='coolwarm')
plt.title("Correlation Between Numerical Variables")
plt.show()
```

